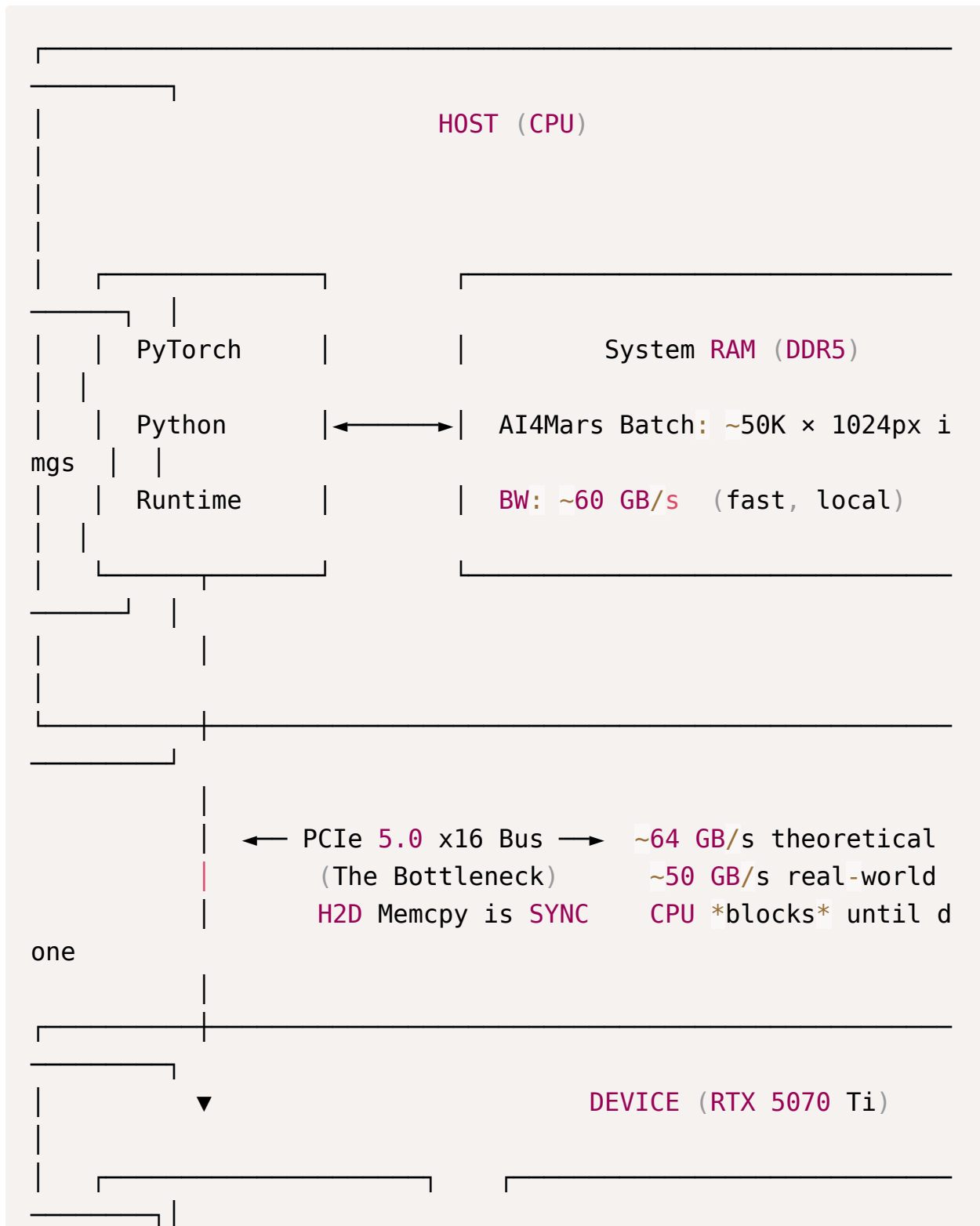
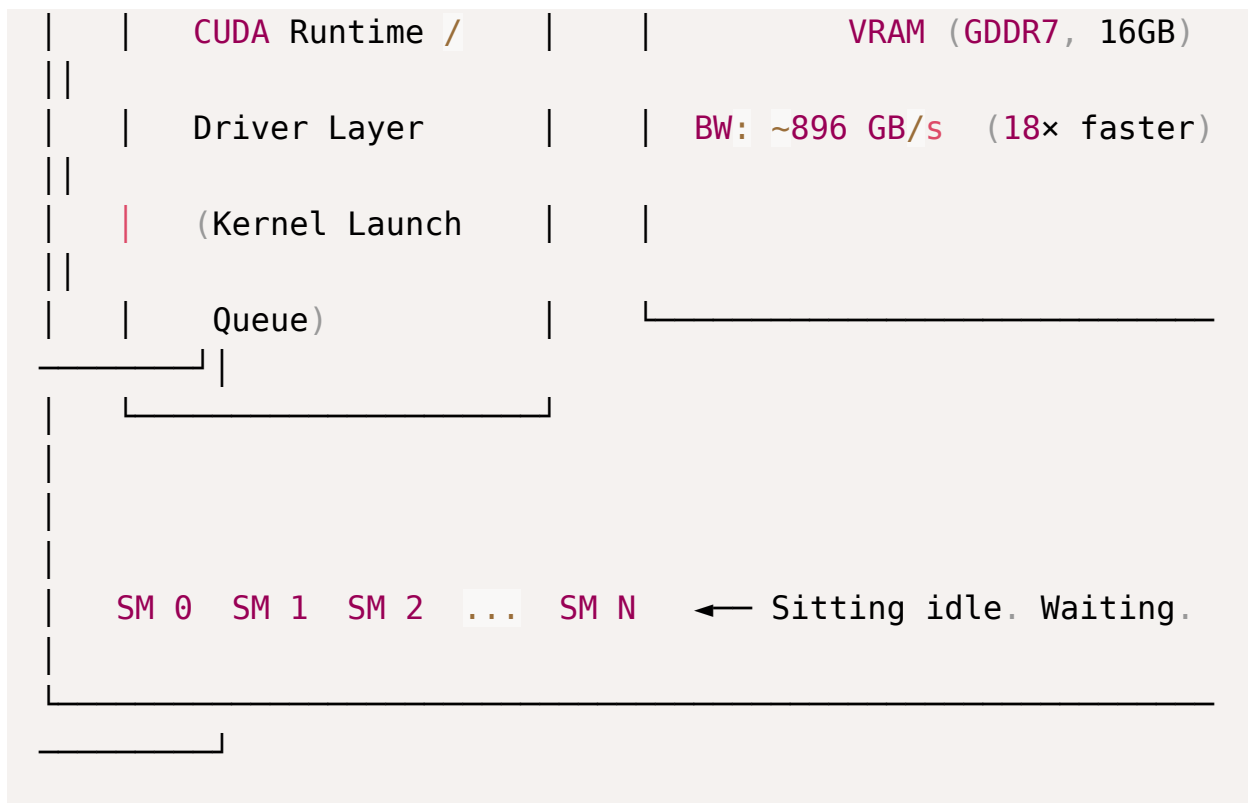


baseline





the two killers of the gpu utilization

The PCIe Bus Bottleneck (Communication Latency)

The model processes mars terrain images and each training step requires moving a batch of tensors from **cpu ram** → **gpu vram** before any cuda kernel can run. This is the **Host-to-Device (H2D) memory transfer**.

In PyTorch's default eager mode, this transfer is **synchronous**, meaning the CPU thread calls `tensor.to('cuda')`, and then sits and waits for the Direct memory access engine to finish copying before the python interpreter continues to the next line.

Consider the arithmetic for your setup. A batch of 16 images at 1024×1024 with 3 channels in FP32 is:

$$16 \times 1024 \times 1024 \times 3 \times 4 \text{ bytes} \approx 201 \text{ MB per batch}$$

At a real-world PCIe 4.0 throughput of ~25 GB/s, that transfer takes roughly ~8 milliseconds. Your RTX 5070 Ti, capable of ~1,340 TFLOPS of FP16 compute, executes a U-Net forward pass in perhaps 15–25 ms. That means ~25–35% of

your wall-clock time is the GPU doing absolutely nothing, waiting on the bus. This is the communication-to-computation ratio problem — and for a dataset like AI4Mars with large images, it is severe.

Spec 4.0 x16 (correct)	PCIe 5.0 x16 (wrong)	PCIe
Transfer rate per lane T/s	32 GT/s	16 G
Theoretical unidirectional GB/s	~63 GB/s	~31.5
Real-world H2D throughput 8 GB/s	~52 GB/s	~27-2
Transfer time for 201MB s ← nearly 2×	~3.9ms	~7.3m
GPU idle fraction (of ~30ms) ← much worse	~13%	~24%

$$\text{VRAM BW} / \text{PCIe BW} = 896 / 27.5 \approx 32\times$$

That's a **32× differential** between what the bus delivers and what the GPU can consume

Kernel Launch Overhead (Synchronization Tax)

even once the data is in vram, eager execution imposes another penalty: **per-operation kernel launch overhead**.

every pytorch operation `nn.Conv2d`, `nn.BatchNorm2d`, `F.relu` maps to one or more cuda kernels. in eager mode, the python runtime dispatches these sequentially and synchronously:

Python calls `conv(x)`

The `CUDA` driver enqueues a kernel to the `default` stream

The `CPU` issues a device synchronization barrier – it waits for the `GPU` to confirm the kernel completed

Only then does Python move to `bn(x)`

Repeat

the gpus streaming multiprocessors are idle during steps 2 & 4. this is the **kernel launch overhead**. the encoder has dozens of CONV → BN → ReLU triplets. that overhead accumulates.

The root cause is that eager execution has no **global view of the compute graph**. It cannot see that `conv → bn → relu` could be fused into a single kernel, nor can it prefetch the next batch while the current one is being processed. It is by design, greedy and memoryless.

